PRODUCT DESIGN / SYSTEM ARCHITECTURE

Критический анализ архитектуры дизайн-системы

Pixso × DeepSeek Harness × shadcn/ui
От импортированного UI-кита к управляемому конвейеру макетов и прототипов

Задача: максимальная автоматизация дизайн-системы в Pixso с готовой библиотекой компонентов, токенами, переключаемыми темами, безопасным редактированием и строгой сборкой интерактивных прототипов через LLM.

Зрелость предложенной архитектуры **6,5 / 10**

Авторская экспертная оценка, не результат инструментального тестирования.

ВЫВОД

Стек жизнеспособен, но до промышленной системы не хватает независимого механизма правил, утверждённых паттернов, контроля изменений и доказанной возможности программной записи нативных взаимодействий.

Основа: предоставленный документ «Вставленный Markdown.md» и последующий авторский аудит. Технологические утверждения дополнительно сверены с открытой документацией по состоянию на 31.10.2026.

01. Резюме для принятия решения

Направление выбрано правильно: Pixso остаётся центральной средой проектирования, готовый UI-кит ускоряет запуск, токены обеспечивают темизацию, а DeepSeek Harness способен оркестрировать действия через MCP. Но текущая архитектура слишком полагается на дисциплину модели и недооценивает устройство продуктовых паттернов, версионирование, безопасность и поведенческую часть прототипов.

> **ГЛАВНЫЙ ТЕЗИС**
> **Не нужно учить LLM «красиво рисовать» по инструкции. Нужно построить конструктор, в котором LLM выбирает утверждённые строительные блоки, а строгий исполнитель технически предотвращает нарушения дизайн-системы.**

Что оставить

- Pixso как среду мастер-компонентов, токенов, шаблонов и результативных макетов.
- Официальный Pixso MCP как основной транспорт чтения и изменения файлов.
- DeepSeek Harness как планировщик и оператор, но не как вседозволенный редактор холста.
- shadcn/ui и Obra Community как кандидатов для быстрого пилота.
- Промежуточную спецификацию экрана, манифест компонентов и аудит после сборки.

Что изменить немедленно

- Назначить единственного владельца визуального контракта и публикации версий.
- Убрать свободный доступ LLM к универсальным изменяющим операциям в штатном режиме.
- Ввести слой patterns/templates/flows между компонентами и полноценными экранами.
- Проверить создание нативных связей прототипа отдельным критическим экспериментом.
- Выпускать изменения через черновик, тесты и публикацию; не править мастер-библиотеку напрямую.

Навигация по отчёту

02 — Сильные стороны и риски · 03 — Источник истины · 04 — Контракт LLM и DS Engine · 05 — Паттерны и UX · 06 — Выбор shadcn/Obra · 07 — Токены и темы · 08 — Прототипирование · 09 — QA и безопасность · 10 — Проверка документации · 11 — Целевая архитектура · 12 — План пилота · 13 — Выводы и источники.

02. Сильные стороны и критические риски

В исходном плане уже есть фундамент зрелой системы: разделение токенов по уровням, переиспользование компонентов, manifest, сценарная сборка, постпроверки и контроль области изменений. Но наличие этих элементов ещё не обеспечивает корректную эксплуатацию.

ОБЛАСТЬ	ПРОБЛЕМА	РИСК
Источник истины	Pixso, CSS и manifest могут редактироваться независимо	Критический
LLM-ограничения	Инструкция не запрещает технически нарушить ДС	Критический
Экранные паттерны	Нет достаточного описания UX-композиции и поведения	Критический
Нативный прототип	Не доказана запись событий и переходов через API	Высокий
Импорт кита	Факт импорта принят за гарантию совместимости	Высокий
QA	Проверяются свойства слоёв, но не весь UX и поведение	Высокий
Эксплуатация	Не описаны публикация, версии и восстановление	Высокий

Оценка рисков — экспертное суждение по предлагаемой архитектуре, а не измеренный показатель существующей инсталляции.

03. Источник истины и модель владения

В исходном документе присутствуют три кандидата на роль источника истины: Pixso, CSS-токены из shadcn/ui и автоматически создаваемый ds-manifest.json. «Двусторонняя синхронизация» звучит удобно, но без приоритетов и конфликтной политики порождает рассинхронизацию.

Пример коллизии: дизайнер меняет radius кнопки в Pixso, разработчик — в CSS, агент обновляет тему по старому manifest. Какой результат считается истинным? Что происходит с существующими инстансами?

Рекомендуемая политика

- **Pixso — визуальный авторитет** для мастер-компонентов, схем композиции и их редакторских свойств.
- **Публикуемый DS Release — неизменяемый контракт** конкретной версии: IDs, варианты, правила, токены, паттерны, тесты.
- **Manifest, правила в формате для LLM и CSS/DTCG-экспорт — производные артефакты** версии, а не три независимых канала правок.
- **Изменение кода не переписывает Pixso автоматически.** Оно оформляется как предложение изменения и проходит согласование/миграцию.
- **Редактирование мастер-библиотеки отделено** от ежедневной сборки экранов.

Формат DTCG 2025.10 — разумный стандарт обмена токенами. Он опубликован как стабильный Community Group Report, но не является рекомендацией W3C. Для Pixso потребуется явный адаптер соответствия типов и алиасов; нельзя гарантировать полную совместимость без проверки. [7]

04. Контракт для LLM и ограниченный исполнитель

Manifest и DS.md полезны, но сами по себе не запрещают агенту создать незадокументированную кнопку, отсоединить инстанс или поменять посторонний фрейм. Это происходит потому, что обычная LLM получает текстовые правила и одновременно универсальные средства редактирования.

> **ПРИНЦИП КОНТРОЛЯ**
> **Дисциплина по промпту не равна технической гарантии. Любой заявленный режим «строго по ДС» должен поддерживаться валидатором и ограниченным набором операций.**

Разделение ролей

1. **Планировщик (LLM):** интерпретирует задачу, подбирает утверждённый паттерн, формирует декларативный screen-spec.
2. **Валидатор (без LLM):** проверяет версии, компонентные ключи, варианты, слоты, допустимую вложенность, токены, ограничения модификации.
3. **Исполнитель Pixso:** преобразует проверенную спецификацию в инстансы и разрешённые технические контейнеры.
4. **Аудитор:** сравнивает результат со spec, контролирует побочные изменения, визуальную и структурную корректность.

Свободные вызовы eval_script и apply_design допускаются в отдельном администраторском режиме для развития ДС, но не выдаются обычному агенту как универсальный инструмент сборки. Контроль доступа должен быть реализован на уровне доступных вызовов, а не одной инструкцией «не создавай сам».

Что должен содержать screen-spec

ID шаблона и версия ДС; дерево разрешённых компонентов; ID и варианты; содержимое и локализация; слоты; ограничения раскладки; theme/mode; breakpoint/viewport; состояния; события и переходы; целевой файл/фрейм; список разрешённых изменений; контрольные ожидания QA.

При этом «собирать только из компонентов» нельзя понимать буквально до последнего узла. Layout-контейнеры, обёртки и фреймы допустимы как инфраструктура, но они не должны заменять UI-компоненты или обходить ограничения системы.

05. Не хватает главного: продуктовых паттернов

Токены управляют значениями. Компоненты определяют анатомию элемента. Но даже идеально токенизированная библиотека Button/Input не содержит решения, как устроить авторизацию, ошибки, сохранение сессии, навигацию или адаптивность.

УРОВЕНЬ	ПРИМЕР	ЗОНА ОТВЕТСТВЕННОСТИ
Tokens	color/primary; space/4	Числовые и цветовые значения
Primitives	Button; Input; Icon	Элементы интерфейса
Composites	FormField; SearchBar	Составные элементы и слоты
Patterns	AuthForm; FilterPanel	Сценарии, иерархия, ошибки
Templates	AuthPage; DashboardShell	Композиция и адаптивная структура
Flows	LoginFlow; CheckoutFlow	Поведение и переходы

Сценарий «Создай логин без макета»: если утверждён AuthPage, агент выбирает этот шаблон, задаёт допустимые свойства и собирает экран. Если шаблона нет, система должна либо отказать в «строгой» сборке, либо перевести запрос в режим предложения нового паттерна с последующим утверждением. Неизвестное решение не должно маскироваться под соответствующее ДС.

Дополнительно паттернам нужны UX-спецификации: успешные и ошибочные состояния, доступность, фокус, поля обязательности, длинные локализованные строки, поведение на разных ширинах, пустые состояния, ограничения текстов.

У Pixso уже описана AI-генерация с пользовательской библиотекой, но при отсутствии нужного модуля она может генерировать его семантически. Поэтому встроенный инструмент не обеспечивает необходимого запрета на «отсебятину». [3]

06. Библиотека: shadcn/ui и Obra

Почему shadcn/ui всё ещё разумный выбор

shadcn/ui полезен не «одинаковыми названиями кнопок» в Figma и React, а контролируемыми исходниками компонентов, семантическими CSS-токенами, расширяемостью и развитой экосистемой. В официальной документации рекомендуются CSS-переменные для темизации, включая пары background/foreground. [6]

Слабое место исходного тезиса: совпадение имён Button, variant, size не создаёт автоматически контракт поведения. Hover, focus, loading, disabled, ARIA, обработчики событий и адаптивная раскладка требуют отдельного описания и проверки.

Что подтверждено про Obra Community

Текущая карточка продукта указывает 57 компонентов shadcn/ui в стиле Nova, 1800+ иконок Lucide, переменную структуру, а среди возможностей Pro — Dark Mode, дополнительные компоненты и блоки. Ранее опубликованный блог описывал другой состав: в этой части следует ориентироваться на текущую карточку и проверять сам скачанный файл. [4]

Лицензия Community — CC BY 4.0: коммерческое использование допускается при выполнении требований атрибуции, указания ссылки на лицензию и изменений. Лицензия Pro — отдельная;

нельзя автоматически переносить условия Community на Pro. [5]

ВАРИАНТ	СИЛЬНАЯ СТОРОНА	РЕШЕНИЕ
Obra Community	Бесплатно проверить анатомию, вариативность, импорт и переменные	Начать пилот
Obra Pro	Больше компонентов/паттернов, темы и поддержка	Покупать после успешного импорта и расчёта экономии
Другой Figma/Pixso kit	Может иметь лучший скелет или прямую поддержку Pixso	Рассматривать, если Obra провалит структурные тесты

Решающие критерии кита

- Сохранение master instance отношений после импорта.
- Сохранение component sets, свойств, slots/instance swap, auto-layout.
- Сохранение переменных с modes и именно их привязок, не только визуальных значений.
- Совместимость анатомии с выбранной веткой компонентов shadcn (включая различия Radix/Base UI).
- Возможность программно извлечь структуру и создать новые корректные инстансы.
- Управляемые лицензия, обновления и версия импортированного исходника.

Pixso официально поддерживает импорт .fig из локального файла и по ссылке, но рекомендует проверять шрифты, структуру, связи мастер-компонентов, порядок слоёв и прототипные переходы после импорта. Импорт не означает автоматически корректную миграцию дизайн-системы. [2]

07. Темизация: что меняется токенами, а что нет

Разделять следует не только primitive/semantic/component, но и оси изменчивости: theme, brand, density, radius, typography. Для каждой оси требуется определить область применения и правила проверки.

НАСТРОЙКА	МЕХАНИЗМ	ОГРАНИЧЕНИЕ
Light / Dark	Режимы семантических цветов	Нужны все интерактивные и статусные состояния
Brand	Алиасы семантики на палитры	Проверка контрастности обязательна
Radius	Числовые токены или режимы	Не меняет структуру и анатомию
Density	Шкалы размеров и отступов	Может приводить к коллизиям и обрезке
Typography	Стили и проверка метрик	Переносы и высота могут меняться
Responsive	Правила контейнеров и шаблонов	Не заменяется одним режимом цвета

В Pixso Plugin API документирована установка явного режима коллекции на узле с помощью setExplicitVariableModeForCollection. Это техническое основание для управления темой на контейнере, но фактическое распространение на импортированные инстансы нужно проверить. [8]

Генератор темы из одного seed-цвета допустим как утилита, но не как гарантия доступности. Контраст текста, outline focus, статусные поверхности, видимость disabled и устойчивость на разных фонах надо проверять отдельно.

08. Нативное прототипирование: главный технический риск

Исходный план предполагает, что Harness через eval_script сможет записать реакции между фреймами и получить интерактивный Pixso-прототип. Сейчас это следует рассматривать как гипотезу, а не подтверждённую возможность.

У Pixso Plugin API свойство reactions у компонента описано типом ReadonlyArray<Reaction>. Наличие модели данных реакций не доказывает наличие публичного изменяющего API. В проверенных материалах не найден достаточный пример программной записи реакций с воспроизводимым результатом. [9]

Две независимые дорожки

A. Нативный Pixso-прототип. Целевой пользовательский артефакт: переходы экранов, модальные окна, состояния, события, демонстрация в Pixso. Обязательно провести отдельный тест записи и проигрывания.

B. Функциональный React-прототип. Отдельная реализация для реального ввода, сложных условий, сетевых данных и исполнения кода. Библиотека shadcn здесь выгодна, но результат нельзя автоматически считать идентичным Pixso.

> **GO / NO-GO**
> **Пока не доказана запись и воспроизведение нативных переходов, этап ds-prototype должен иметь статус «исследование», а не «готовая возможность».**

09. Контроль качества и безопасное редактирование

Один query_all_unique_props не проверяет корректность ДС. Цвет может вручную совпадать с текущим primary, но не иметь привязки к переменной и перестать обновляться при смене режима. Поэтому контроль нужен на уровне ссылок, а не только совпадения значений.

Четыре уровня QA

1. **Структура:** допустимые компоненты и инстансы; нет detached-мастеров; разрешённые слоты; корректная вложенность и версии.
2. **Токены:** действительные переменные/стили, отсутствующие алиасы, корректные режимы, нет запрещённых «сырых» значений.
3. **Визуальная компоновка:** overflow, выравнивание, обрезка, плотность, длинные строки, мобильная ширина, скриншотная регрессия.
4. **Поведение и UX:** состояния form, focus, disabled, error, пустые данные, навигация, доступность и прототипные переходы.

Безопасная транзакционная модель

Рабочий процесс: стабильный релиз изолированная рабочая копия проверка scope применение плана структурный и визуальный diff тесты публикация или отказ. Неизменяемый исходный релиз никогда не является целевым холстом для рискованного массового скрипта.

Pixso документирует, что при ошибке eval_script уже применённые изменения могут не откатываться. Следовательно, нужна изоляция и восстановление, а не упование на обычный Undo. [1]

Нужен журнал изменений

Минимальные поля: время, инициатор, версия ДС, входная команда и нормализованный spec, файл/фрейм, затронутые node IDs, разрешённые и фактические операции, snapshots, статус проверок, причины отказа, ссылка на новую версию/черновик.

10. Сверка возможностей инструментов

Ниже разделены документированные функции и технические предположения. Факты относятся к опубликованной документации на дату отчёта; доступность в конкретном аккаунте зависит от прав и версии клиента.

ВОЗМОЖНОСТЬ	СТАТУС / ОГОВОРКА
Редактирование через MCP	Подтверждено: apply_design, eval_script, write_variables и др. [1]
Local / Remote MCP	Оба могут читать и писать; состав команд не полностью одинаков. [1]
create_instance	В документации указан как Local MCP инструмент. [1]
set_bound_variables	В документации указан как Local MCP инструмент. [1]
Скрипты и layout QA	eval_script, check_layout, screenshots — документированы. [1]
Режим коллекции на frame	Документирован Plugin API, нужен тест на импортированном ките. [8]
Программная запись reactions	Пока не подтверждена публичным примером записи. [9]

ВОЗМОЖНОСТЬ	СТАТУС / ОГОВОРКА
Импорт .fig	Поддерживается; сохранность структуры требует валидации. [2]
D2C+ без Enterprise	Указаны ограничения enterprise space/seat. [10]
MCP в DeepSeek Harness	Подключение Streamable HTTP/stdio задокументировано. [11]
Obra Community / Pro	57 компонентов Community на текущей странице; Pro расширяет набор. [4]

Важная практическая оговорка: Local MCP требует активного открытого файла в десктопном Pixso; стандартный адрес 127.0.0.1:3667 не всегда фиксирован. Remote MCP допускает подключение к облачному файлу с авторизацией, но реальная поддержка нужной цепочки команд проверяется отдельно. [10]

11. Целевая архитектура

Не новая платформа вместо Pixso, а небольшая прослойка контроля между LLM и редактором.



Почему именно так: можно менять LLM без полной переработки правил; хранить один опубликованный визуальный контракт; делать сборку воспроизводимой; проверять scope, версии и разрешённые изменения; предотвращать изменения в мастер-библиотеке в обычных пользовательских сценариях.

Режимы Harness

ds-sync: считать опубликованный релиз и пересобрать manifest без ручного редактирования его содержимого.

ds-compose: создать экран только из утверждённых шаблонов/паттернов и инстансов.

ds-edit: ограниченно изменить узлы и проверить отсутствие побочных диффов.

ds-prototype: построить flow-spec; выполнять запись только после подтверждения API.

ds-audit: структурная, токенная, визуальная и поведенческая проверка.

ds-admin: отдельный, защищённый режим управления мастер-компонентами и миграциями.

12. Пилот: минимальный вертикальный срез

Не стоит импортировать всю библиотеку и сразу строить десятки экранов. Нужна проверка одного сквозного сценария с осмысленными отрицательными тестами.

ШАГ	ЧТО ПРОВЕРЯЕМ	КРИТЕРИЙ ПРИЁМКИ
1	Import Obra Community	Мастера, варианты, инстансы, auto-layout, шрифты сохранены
2	Theme / tokens	Две палитры и Light/Dark переключаются без ручных замен
3	Manifest / contract	Button, Input, FormField доступны со строгими вариантами
4	Создание LoginPage	Экран соответствует утверждённому AuthPage
5	Проверка запретов	Неизвестный вариант отклонён без изменения файла
6	Правка по scope	Ничего вне целевого фрейма не поменялось
7	Нативный flow	Login Error Success реально проигрывается
8	Идемпотентность	Повторная сборка из того же spec эквивалентна

Отрицательные тесты, которые нельзя пропускать

- Несуществующий componentKey или устаревший ID.
- Неподдерживаемое значение variant или попытка вложить компонент в запрещённый слот.
- Токен удалён, режим не существует либо применяется не к тому типу свойства.
- Ошибка в середине нескольких операций редактирования.

- Попытка правки опубликованного мастера из режима ds-edit.
- Отрыв инстанса, нарушение размера кнопки, обрезка кириллического текста.
- Изменение файла другим пользователем между чтением и записью.

Порядок работ после пилота

1. Закрепить пакет договорённостей: владелец, схема версий, именование, токены, компоненты, правила изменения.
2. Расширить библиотеку ровно под реальные сценарии продукта, а не по принципу «добавим всё».
3. Добавить следующую группу проверенных паттернов и шаблонов.
4. Сделать темы, брейкпоинты и UX-проверки обязательной частью CI-подобного аудита.
5. Подключить кодовый прототип как независимый канал только при возникновении подтверждённой потребности.

13. Итоговое решение

Стек оставляем: Pixso + официальный MCP + DeepSeek Harness + shadcn/ui/Obra. От сторонних read-only мостов на первом этапе можно отказаться: они не решают ключевую задачу надёжной управляемой записи.

Архитектуру меняем: утверждённый DS Release, паттерны и шаблоны, отдельный DS Engine, scope-lock, журнал и обязательные тесты. Не смешиваем права на создание интерфейсов с правами на изменение библиотеки.

Самый опасный неподтверждённый участок: программная запись нативных Pixso prototype reactions. Именно он должен получить высокий приоритет в первом техническом эксперименте.

> **РЕКОМЕНДАЦИЯ**
> **Цель проекта — не «ИИ рисует в Pixso», а управляемый конструктор продуктовых интерфейсов, в котором ИИ понимает задачу, но не может обойти утверждённые правила проектирования.**

Граница анализа. Этот документ — архитектурная экспертная оценка и проверка открытой документации. В реальном файле Pixso, в установленном DeepSeek Harness и на конкретном UI-ките инструментальные тесты ещё не проводились. Поэтому выводы о совместимости импорта, полной записи прототипов и стабильности end-to-end сборки являются гипотезами до прохождения пилота.

Источники и документы для технической проверки

В таблице ниже — первичные страницы документации и текущие страницы поставщиков. Ссылки активны. Доступность инструментов в конкретном окружении необходимо подтвердить практическими тестами.

[1] Pixso Developer — MCP Tools & Commands

Список MCP-инструментов и разграничение Local/Remote; операции чтения, записи, проверки и скриптов.

[2] Pixso Help — Import design files into Pixso

Поддерживаемый импорт .fig, требования к проверке шрифтов, структур и компонентных связей.

[3] Pixso Help — Using custom design systems with AI

Возможности встроенной генерации по дизайн-системе и поведение при отсутствующих модулях.

[4] Obra shadcn/ui — Community Edition

Актуальная комплектация Community, варианты расширения Pro и условия тестовой версии.

[5] Obra — Community и Pro license

Условия коммерческого использования Community и отличия лицензии Pro.

[6] shadcn/ui — Theming

Модель семантических CSS-токенов, темы и соглашения по именованию.

[7] Design Tokens Community Group — Technical Reports 2025.10

Опубликованная спецификация формата токенов и её статус.

[8] Pixso Developer — FrameNode / variable modes

Переключение режимов переменных на контейнере и привязка полей.

[9] Pixso Developer — ComponentNode / reactions

Программная модель реакций: чтение документировано, запись требует доказательства.

[10] Pixso Developer — About Pixso MCP / Local MCP

Сопоставление Local и Remote MCP, требования к окружению и правам.

[11] DeepSeek Harness — MCP subsystem documentation

Подключение MCP-клиентов и scope инструментов в DeepSeek Harness.

Оригинальная постановка задачи и история выбора кита приведены в пользовательском документе «Вставленный Markdown.md». Здесь использованы только относящиеся к проекту положения, дополненные независимой архитектурной оценкой и проверкой публичных источников.